



Improving Counting Sort Algorithm Via Data Locality

Shamsed Mahmud
University of Mississippi Medical Center
Jackson, Mississippi, USA
smahmud@umc.edu

Sardar Anisul Haque
Alcorn State University
Lorman, Mississippi, USA
sahaque@alcorn.edu

Nazim Choudhury
University of Wisconsin
Green Bay, Wisconsin, USA
nazimahmed764@gmail.com

ABSTRACT

In this paper, we proposed a cache-friendly hybrid sorting algorithm that combined a non-comparison sorting algorithm (counting sort) with a comparison sort (quick sort) algorithm. The current study leverages the principle of locality to improve the performance of the counting sort algorithm over a large list of integer inputs. We employed a modified version of the original quick sort algorithm to reduce the number of cache misses in counting sort. We also empirically tested the performance of the proposed hybrid algorithm by varying both the range and the quantity of the input values. This hybrid approach not only demonstrated a superior performance over the classic counting sort algorithm but also is capable of facilitating parallelism that was impractical for the classic counting sort algorithm.

CCS CONCEPTS

• **Theory of computation** → **Divide and conquer; Algorithm design techniques; Sorting and searching.**

KEYWORDS

Non-comparison Sort, Counting Sort, Quicksort, Locality, Hybrid Algorithm

ACM Reference Format:

Shamsed Mahmud, Sardar Anisul Haque, and Nazim Choudhury. 2022. Improving Counting Sort Algorithm Via Data Locality. In *2022 ACM Southeast Conference (ACMSE 2022)*, April 18–20, 2022, Virtual Event, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3476883.3520203>

INTRODUCTION

Sorting algorithms have been widely researched for decades [8, 11] due to the ubiquitous need for sorting in many application domains [6, 12, 14]. The primary purpose of these algorithms is to arrange (i.e., order) an array (i.e., list of items) of input data. They can be divided into two dominant categories: (i) comparison based and (ii) non-comparison based sorting algorithms. The comparison based sorting algorithms (e.g., insertion sort, merge sort, quicksort and heap sort etc.), use a comparator function that compares the input values. The time complexity of comparison based sorting algorithms depends on the number of times such comparator function is called.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ACMSE 2022, April 18–20, 2022, Virtual Event, USA

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-8697-5/22/04...\$15.00

<https://doi.org/10.1145/3476883.3520203>

The quick sort algorithm simulates a divide-and-conquer approach and divides an array (i.e., collection) of input values into sub-partitions (i.e., sub-arrays) by selecting a pivot element (i.e., an element selected from the array items). Note that, the words "array" and "partitions" will be used interchangeably to denote a collection of input values in the rest of the study. The selection of the pivot element is non-trivial towards the performance of the quick sort. The elements less than the pivot are kept on the left side whereas the elements greater than or equal to the pivot are kept on the right side. In contrast, the non-comparison sorting algorithms exploit the internal features of the input data to rearrange the values. For example, the counting sort Algorithm 1 works like a hash function that counts the number of objects that have unique key values and applies prefix sum to each count to determine the position of that key value in the output sequence. Due to its linear running time, the number of items (n) and difference between the highest and the lowest key value (i.e., range r) govern the running time. Therefore, this sorting algorithm is suitable for the cases where the variance (i.e., range) of the keys and the number of inputs values are both asymptotically in the same order. Often, it is added as a subroutine in radix sort since it can handle large keys more efficiently [3, 9]. The ideal case for counting sort is where the inputs are sorted either in ascending or descending order (i.e., successive values in close proximity). In contrast, the worst condition for the counting sort is where the inputs are rather scattered or not sorted at all. The memory access pattern of the auxiliary arrays in counting sort (i.e., *count* and *output* in Algorithm 1 depends on the integer keys (i.e., inputs), and the total access time varies for different arrangements of the same input values. Classical Counting sort has the complexity of $O(n + r)$.

Current study proposed a cache-friendly, and memory-efficient hybrid sorting algorithm that combines the quick sort and the counting sort algorithms. The objective of the proposed algorithm is to provide the ideal condition for counting sort via data locality - keeping data in contiguous memory in the order that they are referenced. A modification of the quick sort algorithm is proposed to induce data locality via a preprocessing step and thereby reduced the number of cache misses and increased the amount of cache hits in counting sort. These, in turn, contributed towards the improved execution time of the counting sort via best case. The recursive partitioning of a sub-partition by the modified quick sort was ceased when both the range of the values and the total number of inputs in the sub-partition is less than a threshold value. The main contribution of this research can be summarized as follows: (i) we proposed a novel cache-friendly hybrid sorting algorithm that combines the counting sort and a modified version of the quick sort algorithm, and exploited the data locality concept to exhibit high cache-hit ratio to improve the performance of counting sort, (ii)

we demonstrated the performance improvements by considering two different input sets (a) input values where both range (r) and the number of keys (n) are large, and (b) input values where the range of the keys is larger ($r \gg n$) than the number of integer keys. The impact of this research is two-fold: (i) firstly, it provides better execution time for a non-comparison sorting algorithm and (ii) secondly, it facilitates parallel execution of the counting sort algorithm which is generally inconceivable for the classic version.

1 CACHE COMPLEXITY OF COUNTING SORT AND MOTIVATION

Algorithm 1: Counting Sort Algorithm

```

1 void countingsort(int arr[], int n) {
2     int output[n+1];
3     // getMax() returns the greatest value from $arr$
4     int r = getMax(arr, n);
5     int count[r+1];
6     for(int i = 0; i <= r; i++)
7         count[i] = 0;
8     for(int i = 0; i < n; i++)
9         count[arr[i]]++;
10    for(int i = 1; i <= r; i++)
11        count[i] += count[i-1];
12    for(int i = n-1; i >= 0; i--) {
13        output[count[arr[i]]] = arr[i];
14        count[arr[i]] -= 1;
15    }
16    for(int i = 0; i < n; i++)
17        arr[i] = output[i];
18 }
```

We first analyzed the cache complexity of the counting sort algorithm using the *Ideal Cache Model* [4]. The Algorithm 1 employed a hashing technique by counting the number of occurrences of different input values. For an input array of length n non-negative integers within the range $[0, r - 1]$, the estimated total number of cache misses, according to the ideal cache model is at most $3n + 4n/L + 2r/L + 6$ (L = length of the cache line that represents the unit of data transfer between the cache and the main memory). Here both n and r are small enough such that neither of the input array (i.e., $arr[]$) nor the auxiliary arrays (i.e., $count[]$, $output[]$) can be stored into the cache entirely [5]. The total number of cache misses for each array access in the Algorithm 1 is described below. The linear access of the $count[]$ array in lines 6-7 and 10-11 results in $2r/L + 2$ cache misses. Despite the linear access of the input array $arr[]$ in line 8-9, the non-linear access of the auxiliary array $count[]$ results in $(n/L + 1) + n$ cache misses in the worst case. Although the details of the conflict and the capacity misses are out of the scope of this study, however, interested readers can learn about these in [2]. The $output[]$ array is populated in lines 12-14 which also involve accessing all the arrays (i.e., $arr[]$, $output[]$ and $count[]$). In this case, like before, despite the linear access of the $arr[]$, the non-linear n accesses to the auxiliary arrays (i.e., $count[]$ and $output[]$) result in a total of $(n/L + 1) + 2n$ cache misses in the worst case. Again, the latter $2n$ cache misses are due to both the capacity and the conflict misses. Finally, the linear access of both

$arr[]$ and $output[]$ in lines 15-16 results in $2n/L + 2$ cache misses. The aforementioned estimates of the cache misses consider the fact that every access to $output[]$ or $count[]$ array can result in a cache miss. It is evident that a counting sort algorithm has scopes for improvement via the employment of data locality and thus reducing the cache misses and improving the cache hits. Therefore, we drew our primary motivation from the study by Lamarca and Ladner [10] where the authors advised in favor of data locality and stressed on developing techniques to improve locality even if it results in increasing number of executed instructions.

2 PROPOSED MODEL

In this section, we first demonstrate a simple experiment to justify the rationality of our current study. We then describe our proposed algorithm followed by the empirical results.

2.1 Scope of the Current Studies

Two simple experiments were performed to understand the impact of the ordering of input values on counting sort algorithm. We im-

Table 1: Performance Comparison in ms Over Random (T_1) and Sorted Inputs (T_2)

n	r	T_1	T_2
1000000	1000000	1.09	0.467
2000000	2000000	2.84	0.910
3000000	3000000	3.75	1.39

plemented the classic counting sort algorithm, given in Algorithm 1, on an *Intel(R) Core(TM) i7-7500U CPU @ 2.70GHz* machine that has 32K of L1 data cache, 32K of L1 instruction cache and 256K of L2 cache. In our first experiment, n input integers were drawn independently at random between $[0, (n - 1)]$. In the other experiment, the input integer array was already sorted, that is, $\{0, 1, \dots, n - 1\}$. The execution times from both experiments for different values of n and r (i.e., range of input integers) are given in Table 1. Theoretically, the time complexity for both experiments should have been similar $O(n)$. It becomes a linear time algorithm when the range of the input values is equal to the total number of input values ($n = r$). However, in practice, we found that for the same number of input values n , the counting sort algorithm runs 2 – 3 times faster for an input of a sorted array than a random array (i.e., T_2 in Table 1). This performance improvement can be explained by the principle of locality. The locality of reference is implemented to utilize the full benefit of cache memory in computer organization.

The access pattern of the auxiliary arrays ($count[]$ or $output[]$) is random and can produce one cache miss per access in the worst case. One of the limitations of the Random Access Machine (RAM) model [7] is that it cannot explain the time variance contributed by the cache misses/hits. From Table 1, this limitation is evident where we observe that for random integers, the execution time did not scale linearly (as expected theoretically) due to poor cache utilization. For example, the average CPU execution time for sorting 300000 random integers (i.e., 3.75 ms) is not one third of the execution time to sort 100000 sorted integers (i.e., 1.09 ms). Similar experimental results were also reported in other studies [5, 13].

For the first experiment (random input), according to the ideal cache model, the estimated cache misses is still $3n + 4n/L + 2r/L + 6$. In the second experiment (sorted input), the algorithm executed without any conflict or capacity cache misses. Thus, the resulting cache misses remained $4r/L + 5n/L + 9$. This experimental results suggest that the performance of counting sort algorithm can be improved via data locality. The intent here is to accelerate memory access by arranging data to take the advantage of CPU caching.

2.2 Algorithms

In the proposed algorithm, we employed the quick sort algorithm as a preprocessing step for the counting sort algorithm. The objective was to rearrange the input array $arr[0, \dots, n-1]$ and generate $i > 0$ partitions (i.e., $\{arr[0, p_1-1]\}, \{arr[p_1, p_2-1]\}, \dots, \{arr[p_i-1, n-1]\}$) satisfying the following conditions: (i) any integer in partition $\{arr[p_j, p_{j+1}-1]\}$ is smaller than all integers in partition $\{arr[p_k, p_{k+1}-1]\}$, for $0 \leq j < k \leq i$, and (ii) the sum of the range and the number of integers in any part is equal or smaller than a threshold value $C > 0$. If the input array is partitioned as described above for a reasonable value of C (explained later), the access pattern of the auxiliary arrays in each iteration of counting sort algorithm (Algorithm 1) becomes cache friendly. Because, for each part of the input array, the accesses to the (i) $count[]$ is limited by the range (i.e., r_i) of that partition, and (ii) $output[]$ is limited by the number of integers (i.e., n_i) in that partition.

We achieved the above partitioning via a modified version of the quick sort given in Algorithm 2. In this recursive algorithm, the sub-partition of the input array $arr[low, \dots, high-1]$ is partitioned by a separate routine called *partition()*. We implemented a popular partitioning method called *median of three* [1]. In this sub-partition, *maxValue* and *minValue* are the highest and the lowest integer value respectively. The modified version of the quick sort suspends any recursive calls further once the sum of the range of the values in the sub-partition and the number of values in the sub-partition became smaller than the threshold value C . We consider C as an algorithmic parameter and the significance of this parameter is described in the next subsection.

Algorithm 2: Modified Version of Quicksort Algorithm Used as a Pre-processing Step

```

1  void quicksort_modified(int arr[], int low, int high,
2                          int maxValue, int minValue) {
3      while ((low < high) &&
4             (maxValue - minValue +
5              high - low > C))
6      {
7          int pivot = partition(arr, low, high);
8          int midValue = arr[pivot];
9          quicksort_modified(arr, low,
10                           pivot - 1, midValue,
11                           minValue);
12          quicksort_modified(arr, pivot + 1,
13                           high, maxValue,
14                           midValue);
15     }
```

2.3 Correctness of the Proposed Algorithm

The original version of the counting sort algorithm remains unchanged, however, the quick sort algorithm was modified in the preprocessing step of the proposed method. Let us consider the sub-array $arr[p_j, \dots, p_{j+1}-1]$ be the j^{th} part of the input array as described in sub-section 2.2, and max and min be the maximum and minimum value respectively from this sub-array. Then $count[min, \dots, max]$ would be the corresponding sub-array responsible for keeping the frequencies of integers in $arr[p_j, \dots, p_{j+1}-1]$. The preprocessing step in Algorithm 2 guarantees that $p_{j+1} - p_j + (max - min) \leq C$. The sorted integers of this sub-array will eventually be placed in sub-array $output[p_j, \dots, p_{j+1}-1]$ according to the Algorithm 1. In this algorithm, unlike $arr[]$, the access pattern for both $count[]$ and $output[]$ are irregular. Therefore, due to the adoption of the optimal cache replacement policy, the ideal cache model is capable of organizing enough room for C inputs for the sorting of j^{th} sub-array. This arrangement avoids any conflict-/capacity cache misses in accessing both $count[min, \dots, max]$ and $output[p_j, \dots, p_{j+1}-1]$. We claim that the estimation of the cache misses can actually be realized depending on the estimation of the value of parameter C which is dependent on the computer architecture. Note that we considered the ideal cache model as the standard to estimate the cache-misses. Therefore, the suboptimality is guaranteed in the proposed algorithm. However, in our experiments, we considered $C = 1000$.

2.4 Cache Complexity of Our Proposed Algorithm

As a preprocessing step, the modified quick sort procedure (Algorithm 2) arranges the input array into a number of partitions. The number of cache misses by the counting sort procedure (Algorithm 1) for this processed input array is estimated below.

The access pattern of the auxiliary array $count[]$ remains linear both in *lines 6-7* and *lines 10-11*. It causes $2r/L + 2$ cache misses. The linear access of $arr[]$ causes $n/L + 1$ cache misses in *lines 8-9*. For each partition of the input array (i.e., $arr[]$), the corresponding partition of $count[]$ is brought into cache once. Therefore, no conflict or capacity cache misses for the sub-arrays were observed. Three different observations may arise depending on the size of r and n . Firstly, when the difference between r and n is insignificant, the estimated total number of cache misses would be $r/L + i$ for i partitions of the input array $arr[]$. Secondly, when $r \gg n$, the estimated cache misses would be n for accessing $count[]$ because the associated partitions of $count[]$ for the input sub-array would have fewer non-zeros. Finally, when $r \ll n$, the estimated cache misses would become $r/L + 1$ for accessing $count[]$. This is because the associated partitions of $count[]$ for the input sub-array was expected to be very small and we can expect a linear access of this array. *Lines 12-15* involved accessing three arrays namely, $arr[]$, $output[]$ and $count[]$. The linear access of array $arr[]$ causes $n/L + 1$ cache misses. The total number of cache misses for accessing $count[]$ is $r/L + i$ or n or r/L due to the same three aforementioned observations. Since the sorted order of j^{th} partition $\{arr[p_j, p_{j+1}-1]\}$ of the input array $arr[]$ will be available in the corresponding partition of $output[p_j, p_{j+1}-1]$, the resulting cache misses for accessing $output[]$ was $n/L + i$. Note that the sorted output in j^{th} partition of

Table 2: Running Times in ms for Counting Sort with and without Preprocessing

n	r	T_1	T_2
1000	1000000	0.02+1.28	1.51
2000	1000000	0.06+1.41	1.57
3000	1000000	0.1+1.4	1.56

$output$ corresponds to the j^{th} partition of $input$. Finally, the linear access of both $arr[]$ and $output[]$ in lines 16-17 resulted in $2n/L + 2$ cache misses.

By considering the number of partitions less than the total number of inputs (i.e., $i < n$), for three different cases, we summarize the expected improvement of the proposed algorithm. This improvement is due to the preprocessing step that results in the reduced amount of cache misses. Firstly, for the insignificant difference between n and r , the improvement is $3n - n/L - 2r/L - 3i$ cache misses. Secondly, for $r \gg n$, the expected improvement is $n - n/L - i$. Finally, for $r \ll n$, the expected improvement is $3n - n/L - 2r/L - i$. In all cases, positive improvement of the cache misses was evident.

3 EXPERIMENTAL RESULTS AND ANALYSIS

To test the performance of the counting sort with proposed preprocessing step, we experimented two cases: (i) where n and r are in the same range (i.e., the favourable/best case condition for the counting sort), and (ii) where $r \gg n$ - the worst case scenario for counting sort algorithm as the time complexity depends on r significantly. In the first case, we executed three algorithms: (i) original quick sort algorithm, (ii) the proposed algorithm, and (iii) a hybrid algorithm with quick sort and insertion sort. The reason to consider the hybrid algorithm combining the quick and insertion sort is to demonstrate the performance improvement of the proposed algorithm over another hybrid algorithm. In this hybrid algorithm, the insertion sort orders the input of the quick sort when the size of the recursively-defined partition is sufficiently small. In Table 2, we present the running time of the counting sort with the proposed preprocessing step T_1 and without the preprocessing step T_2 . The execution time in T_1 has two parts where the first part denotes preprocessing time (i.e., execution time for the modified quick sort algorithm) and the second denotes the execution time of the counting sort algorithm. It is evident that with the trivial preprocessing time, the proposed algorithm demonstrated improved performance over the classic counting sort algorithm. Although, in this experiment, the range r is far greater than the total number of input integers n which is a non-ideal condition for the counting sort algorithm, however, the performance improvement of the proposed algorithm is promising.

In Table 3, we present the running time of three different algorithms: classic quick sort algorithm (T_1), modified quick sort with insertion sort (T_2), and the proposed algorithm T_3 . When the total number of input integers n is large and $n = r$, our proposed algorithm outperforms the others.

Table 3: Running Times in ms for Quicksort, Quicksort with Insertion Sort, and Quicksort with Counting Sort

$n = r$	T_1	T_2	T_3
1000000	103.34	93.28	76.74
2000000	212.24	195.58	159.38

4 DISCUSSION AND CONCLUSION

In this study, we proposed a cache-friendly, and hybrid sorting algorithm that combines a classic non-comparison sorting algorithm (i.e., counting sort) with a preprocessing phase. The preprocessing phase was addressed by a modified version of a comparison sort (quick sort). The ultimate objective was to improve the execution time of the counting sort via a reduced number of cache misses due to data locality. We also observed non-trivial performance improvement of the counting sort algorithm in both ideal and non-ideal cases. This improvement is critical and the modification of the algorithms has significant impact on arranging large number of input integers where the range of values is considerably higher. Since each partition of the input array can be sorted independently, this supports an opportunity to apply parallelism to counting sort algorithm efficiently. Due to the limitation of the RAM model, it is out of scope to compute asymptotic upper bound for the proposed algorithm however, this proposed algorithm is not free from its limitation. The quick sort algorithm in the preprocessing step jeopardises the stable property of the counting sort algorithm. Future studies can explore various stable sort as a preprocessing step and compare the performance improvement.

REFERENCES

- [1] Jon L. Bentley and M. Douglas McIlroy. 1993. Engineering a Sort Function. *Software: Practice and Experience* 23, 11 (1993), 1249–1265.
- [2] Jamison D. Collins and Dean M. Tullsen. 1999. Hardware Identification of Cache Conflict Misses. In *MICRO-32. Proceedings of the 32nd Annual ACM/IEEE International Symposium on Microarchitecture*. IEEE, Haifa, Israel, 126–135.
- [3] Jeff Edmonds. 2008. *How to Think About Algorithms*. Cambridge University Press.
- [4] Matteo Frigo, Charles E. Leiserson, Harald Prokop, and Sridhar Ramachandran. 1999. Cache-oblivious Algorithms. In *40th Annual Symposium on Foundations of Computer Science (Cat. No. 99CB37039)*. IEEE, New York, USA, 285–297.
- [5] Sardar Anisul Haque. 2013. *Hardware Acceleration Technologies in Computer Algebra: Challenges and Impact*. Ph.D. Dissertation. University of Western Ontario.
- [6] Wu Jianping, Yutang Ye, Liu Lin, Huang Bingquan, and Guo Tao. 2011. High-speed FPGA-based SOPC Application for Currency Sorting System. In *IEEE 2011 10th International Conference on Electronic Measurement & Instruments*, Vol. 2. IEEE, Chengdu, China, 85–89.
- [7] Donald E. Knuth. 1968. *The Art of Computer Programming: Fundamental*. *Algorithms 1* (1968), 187.
- [8] Donald E. Knuth. 1997. *The Art of Computer Programming*. Vol. 3. Pearson Education.
- [9] Donald E. Knuth. 1998. *The Art of Computer Programming: Sorting and Searching*. Vol. 3. Addison-Wesley, Reading.
- [10] Anthony LaMarca and Richard E. Ladner. 1999. The Influence of Caches on the Performance of Sorting. *Journal of Algorithms* 31, 1 (1999), 66–104.
- [11] Tom Leighton, Yuan Ma, and C. Greg Plaxton. 1997. Breaking the $\Theta(n \log^2 n)$ Barrier for Sorting with Faults. *J. Comput. System Sci.* 54, 2 (1997), 265–304.
- [12] Robert Meolic. 2013. Demonstration of Sorting Algorithms on Mobile Platforms. In *CSEDU*. Aachen, Germany, 136–141.
- [13] Naila Rahman and Rajeev Raman. 1999. Analysing Cache Effects in Distribution Sorting. In *International Workshop on Algorithm Engineering*. Springer, London, UK, 183–197.
- [14] Ran Zhang, Xue Wei, and Takahiro Watanabe. 2013. A Sorting-based IO Connection Assignment for Flip-chip Designs. In *2013 IEEE 10th International Conference on ASIC*. IEEE, Shenzhen, China, 1–4.